

1)

[1]:

```
import pandas as pd
```

```
[9]: df = pd.read_csv("C:\\Users\\Admin\\Downloads\\penguins_dataset.csv")
```

```
[10]: df.head()
```

```
[10]:
```

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm
0	Adelie	Torgersen	39.1	18.7	18
1	Adelie	Torgersen	39.5	17.4	18
2	Adelie	Torgersen	40.3	18.0	19
3	Adelie	Torgersen	NaN	NaN	19
4	Adelie	Torgersen	36.7	19.3	19

A)

1. Verify the dataset loaded correctly by ensuring the file path is correct
2. We can understand the structure and format and See how columns appear ,Check column ordering and Confirm that headers were read properly
3. Detect obvious data quality issues early like Missing values ,Strange symbols ,Wrong column types and Misaligned rows

2)

```
[11]: df.shape
```

```
[11]: (344, 7)
```

```
[ ]: |
```

```
[12]: df.columns
```

```
[12]: Index(['species', 'island', 'culmen_length_mm', 'culmen_depth_mm',  
         'flipper_length_mm', 'body_mass_g', 'sex'],  
        dtype='object')
```

```
[ ]:
```

```
[13]: df.dtypes
```

```
[13]: species      object
      island      object
      culmen_length_mm  float64
      culmen_depth_mm  float64
      flipper_length_mm  float64
      body_mass_g      float64
      sex          object
      dtype: object
```

```
[ ]: |
```

A) yes

B) Categorical attributes: Species ,island , sex

Numerical attributes (measured quantities):

culmen_length_mm ,culmen_depth_mm ,flipper_length_mm , body_mass_g

3) missing values

```
[14]: df.isna().sum()
```

```
[14]: species      0
      island      0
      culmen_length_mm  2
      culmen_depth_mm  2
      flipper_length_mm  2
      body_mass_g      2
      sex          10
      dtype: int64
```

```
[ ]: |
```

Calc percentage of missing values.

```
[15]: (df.isna().sum() / len(df)) * 100
```

```
[15]: species      0.000000
      island      0.000000
      culmen_length_mm  0.581395
      culmen_depth_mm  0.581395
      flipper_length_mm  0.581395
      body_mass_g      0.581395
      sex          2.906977
      dtype: float64
```

```
[ ]: |
```

A) Attributes which are most affected by missing values?

1. sex
2. culmen_length_mm / culmen_depth_mm / flipper_length_mm / body_mass_g

B. Impact of missing values for data analysis if left untreated

1. Statistical calculations become inaccurate as Mean, median, standard deviation, and correlations may be biased.
2. Machine learning models fail or perform poorly as many algorithms cannot handle NaN values and missing data can reduce model accuracy and distort predictions.
3. Visualizations become misleading as Scatter plots or histograms may show fewer points, hiding trends and Box plots might appear skewed.
4. Loss of information can happen as dropping rows reduces the dataset size and may remove important patterns.
5. Incorrect conclusions will be made as missingness may not be and this can bias results toward certain species or age groups.

4)

Identify invalid or unexpected values in numerical columns

```
[16]: df[df['body_mass_g'] <= 0]
```

```
[16]: species island culmen_length_mm culmen_depth_mm flipper_length_mm body_mass_g sex
```

```
[ ]: |
```

```
[17]: df[df['flipper_length_mm'] <= 0]
```

```
[17]: species island culmen_length_mm culmen_depth_mm flipper_length_mm body_mass_g sex
```

```
[ ]: |
```

```
[18]: df[df['culmen_length_mm'] <= 0]
```

```
[18]: species island culmen_length_mm culmen_depth_mm flipper_length_mm body_mass_g sex
```

```
[ ]: |
```

```
[19]: df[df['culmen_depth_mm'] <= 0]
```

```
[19]: species island culmen_length_mm culmen_depth_mm flipper_length_mm body_mass_g sex
```

Look for duplicates caused by spacing, case differences, or typos.

```
[20]: df['species'].unique()  
df['island'].unique()  
df['sex'].unique()
```

```
[20]: array(['MALE', 'FEMALE', nan, '.'], dtype=object)
```

```
[ ]: |
```

A)

Syntactic Validity : Checks whether a data value follows the correct format, structure, or pattern, without considering if the value makes sense in the real world : is the value formatted correctly?"

Ex: Correct data type (string, number, date) ,Correct text structure (no numbers inside a name)

Consistent capitalization or spelling

Semantic Validity : Checks whether the data value is logically meaningful and realistic based on domain knowledge.asks Does this value make sense in real life?"

Ex: Biological measurements within possible ranges , Valid species names ,No negative or zero body mass

B)

1. Syntactic Validity : Column: sex ; Values found: ['MALE', 'FEMALE', nan, '.']

2. Semantic Validity: : body_mass_g

If we find values like: 0 or -50 ; These are semantically invalid, because a penguin cannot have zero or negative body mass.

5)

Appropriate Strategies to Handle Missing Data

```
[21]: num_cols = ['culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g']

      for col in num_cols:
          df[col] = df[col].fillna(df[col].median())

[22]: cat_cols = ['species', 'island', 'sex']

      for col in cat_cols:
          df[col] = df[col].fillna(df[col].mode()[0])
```

A)

Used these strategies for missing values instead of removing records to avoid losing valuable data, prevent bias, maintain dataset size, and ensure the dataset remains usable for analysis and modelling. The median is used for numerical attributes due to robustness against outliers, while the mode is appropriate for categorical values because it preserves the most common class.

6)

```
[23]: df = df[df['body_mass_g'] > 0]

[24]: df['sex'] = df['sex'].str.capitalize()
```

```
[ ]:
```

A) Domain knowledge acts as a validation framework that turns raw numbers into meaningful information. It enables analysts to confidently identify invalid data, apply appropriate corrections, and ensure that the dataset accurately reflects real-world phenomena, leading to more reliable analysis and conclusions.

7)

```
[25]: df['mass_group'] = pd.cut(  
      df['body_mass_g'],  
      bins=[0, 3500, 4500, 7000],  
      labels=['light', 'medium', 'heavy']  
      )  
  
[26]: df['culmen_size'] = df['culmen_length_mm'] + df['culmen_depth_mm']  
  
[ ]: |
```

A)

Derived features bridge the gap between raw measurements and meaningful insights. They make data easier to analyse, easier to explain, and often more powerful for statistical analysis and predictive modelling.

08. Create meaningful subsets of the dataset

```
[27]: adelie_df = df[df['species'] == 'Adelie']  
  
[28]: dream_island_df = df[df['island'] == 'Dream']  
  
[ ]: |
```

A. In what situations is sub setting preferred over working with the full dataset?

Sub-setting is preferred when the analysis needs to focus on a specific group, condition, or context rather than the overall population.

It is especially useful when:

1. Comparing specific groups because Analysing only *Adelie* penguins helps identify species-specific traits without interference from others.
2. Reducing complexity because Large or diverse datasets can hide pattern and working with one island or one species makes trends clearer and easier to interpret.
3. Targeted analysis or hypothesis testing : If a research question is island-specific or species-specific, using the full dataset may introduce irrelevant variation.

09. Summarize data using grouping operations

```
[29]: df.groupby('species')['body_mass_g'].mean()
```

```
[29]: species
Adelie      3702.960526
Chinstrap   3733.088235
Gentoo      5067.741935
Name: body_mass_g, dtype: float64
```

```
[ ]: |
```

```
[30]: df.groupby(['species', 'sex'])['flipper_length_mm'].mean().reset_index()
```

```
[30]:
```

	species	sex	flipper_length_mm
0	Adelie	Female	187.794521
1	Adelie	Male	192.037975
2	Chinstrap	Female	191.735294
3	Chinstrap	Male	199.911765
4	Gentoo	.	217.000000
5	Gentoo	Female	212.706897
6	Gentoo	Male	220.876923

```
[ ]:
```

A. How does aggregation change the granularity of the dataset?

Aggregation reduces granularity by summarizing multiple records into fewer representative values.

- Original dataset: One row per individual penguin
- Aggregated dataset: One row per group (e.g., species or species $\times$ sex)

This shifts the focus from individual-level details to group-level patterns, which is useful for comparison and trend analysis.

B. What patterns do you observe across different species?

Typical patterns observed in the Penguins dataset include:

- Gentoo penguins : Highest average body mass , Longest flipper lengths
- Adelie penguins : Lowest average body mass , Shorter flippers
- Chinstrap penguins : Intermediate values between Adelie and Gentoo

When grouped by species and sex:

- Males generally have larger flipper lengths than females within the same species.
- Sexual dimorphism is visible but varies by species.

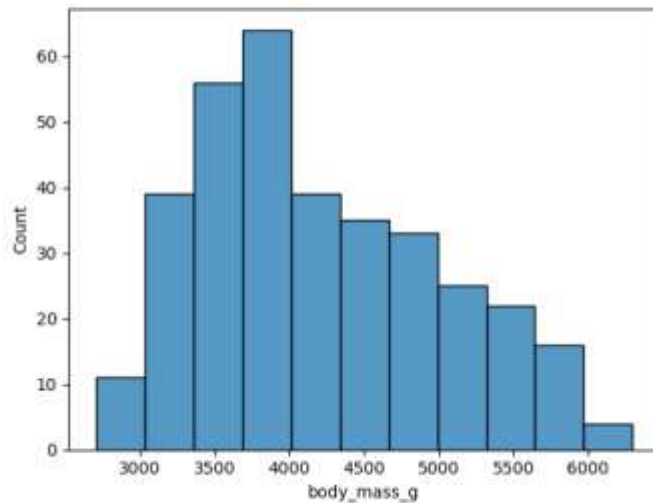
Conclusion:

Species differences are strong and consistent, indicating that species is a major driver of physical variation.

10. Univariate Visual Analysis

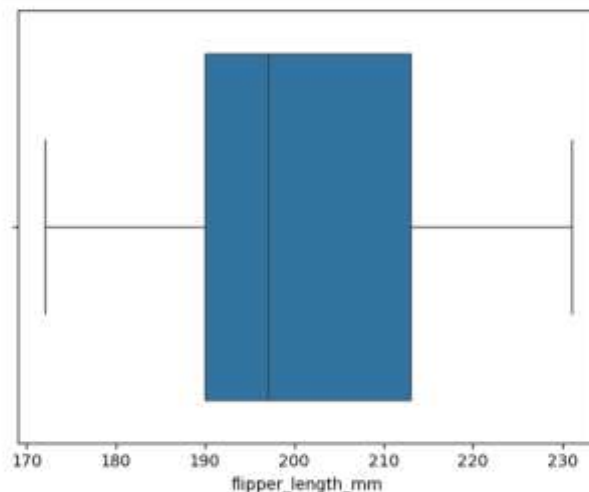
```
[11]: import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df['body_mass_g'])
plt.show()
```



```
[ ]:
```

```
[12]: sns.boxplot(x=df['flipper_length_mm'])
plt.show()
```



A. What does the spread of the data suggest about variability among penguins?

- The body mass histogram shows a wide spread, indicating substantial variability among penguins. This variability reflects: Differences between species, between sexes and Natural biological diversity

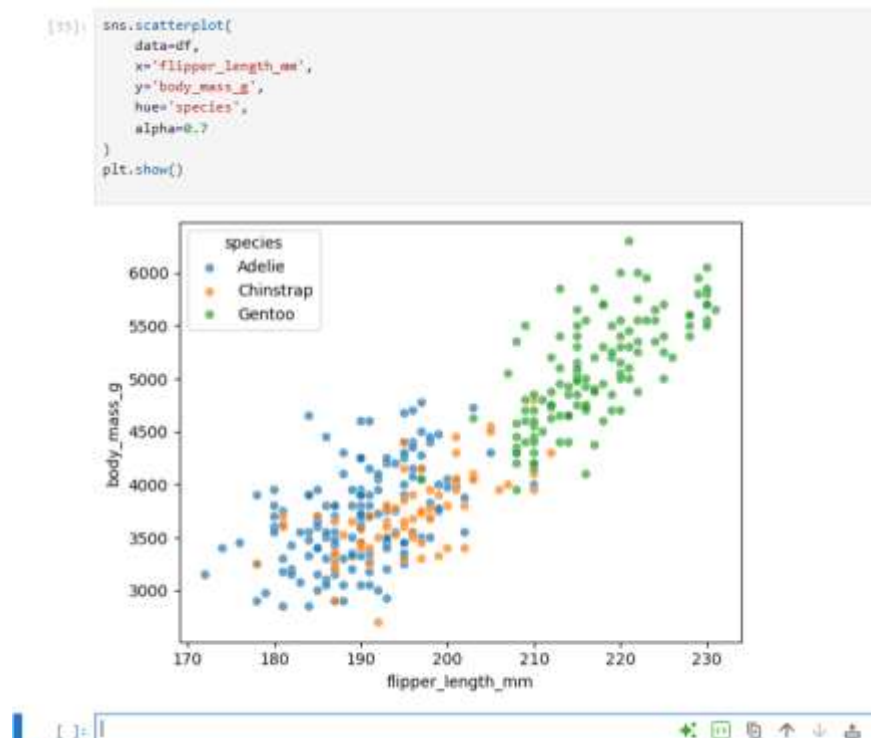
The flipper length box plot further confirms: Species differ significantly in size

- Some groups are more tightly clustered, while others show broader ranges

B. Are there any visible outliers?

Yes, : Body mass (very heavy or very light individuals) , Flipper length (extreme values)

11. Bivariate Analysis



A. Describe the relationship between flipper length and body mass

There is a strong positive relationship between flipper length and body mass: Penguins with longer flippers tend to have higher body mass. The scatter plot shows an upward trend, indicating correlation.

B. Does this relationship appear consistent across species?

The relationship is consistent but species-dependent: All species show a positive trend.

12. Enhance the dataset with additional context and Insights Reflection

```
[34]: from datetime import datetime
      df['processed_date'] = datetime.today().strftime('%Y-%m-%d')
```

```
[35]: df['data_source'] = "Palmer Penguins Dataset"
```

```
[ ]: | 
```

A. Why is metadata important in collaborative data science projects?

Metadata provides context and traceability, which is crucial when multiple people work with the same dataset. It helps to: Track when data was processed, Identify the data source, Ensure reproducibility, Prevent misuse of outdated or incorrect data

B. Which visualization revealed the most interesting pattern, and why?

The scatter plot of flipper length vs body mass was the most insightful because:

- It revealed a clear biological relationship
- It visually separated species into clusters
- It combined size, correlation, and species information in one view

C. How did data cleaning affect the patterns you observed?

Data cleaning:

- Removed invalid or inconsistent values, Reduced noise caused by missing or erroneous records, Made trends clearer and more reliable

As a result:

- Visualizations became smoother
- Group averages became more meaningful
- Outliers were easier to interpret correctly

D. Which steps in this lab align with the refined data stage discussed in lectures?

The following steps directly align with the refined data stage:

- Handling missing values
- Correcting invalid measurements
- Standardizing categorical values
- Creating derived features

